Deep Reinforcement Learning
Module 2: Deep Q-Learning (DQN)

Mustafa Ozger

Aalborg University

March 3, 2026

Where Module 2 fits

- Module 1: MDPs, returns, value functions, Bellman equations.
- Module 2: **Deep value-based RL** for **discrete actions**.
- Goal: learn $Q_\theta(s, a) \approx Q^*(s, a)$ and act (approximately) greedily.

Why we need this

State spaces are often huge (images, sensors, high-dimensional features), so tabular $Q(s, a)$ is infeasible.

Learning outcomes (DQN)

- Connect Q-learning to Bellman optimality.
- Understand Q_θ (linear + MLP) and what parameters θ mean.
- Derive and implement the DQN target/loss with terminal masking.
- Explain why DQN is unstable and how replay + target network stabilize it.
- Evaluate and debug DQN runs properly.

Agenda

1. Q-values $\rightarrow$ Bellman optimality $\rightarrow$ Q-learning
2. DQN idea: replace the table with Q_θ
3. Function approximation (linear + MLP) and what θ, n, k mean
4. DQN loss/target + replay + target network
5. Practical knobs: exploration, loss, optimization (Adam), evaluation, debugging

From Q-learning to DQN

Concepts • Intuition • Implementation

DRL Module 2

What is $Q(s, a)$ in plain language?

Interpretation

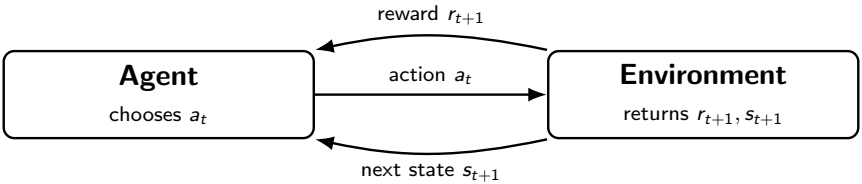
$Q(s, a)$ is the **expected long-term score** if you:

- are currently in state s,
- take action a now,
- then continue (near-)optimally.

Everyday analogy

If s is your location and a is a route choice, $Q(s, a)$ is the expected overall travel quality (time/cost/convenience), not only the next turn.

Agent–Environment loop (source of data)



One transition

$$(s_t,\ a_t,\ r_{t+1},\ s_{t+1},\ \mathit{done})$$

These transitions are the training data for DQN.

Bellman optimality for Q^*

$$Q^*(s,a) = \mathbb{E}\Big[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \,\Big|\, s_t = s, a_t = a \Big].$$

- Immediate reward + discounted best future value.
- Optimal action: $\pi^*(s) \in \arg\max_a Q^*(s,a)$.

Learning goal

We do not know Q^*. We build an estimate from sampled transitions (bootstrapping).

Tabular Q-learning (what we generalize)

Update

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big).$$

- The term in parentheses is the TD error δ.
- Tabular storage breaks when $|\mathcal{S}|$ is huge.

Motivation for DQN

Replace the table with a function approximator Q_θ.

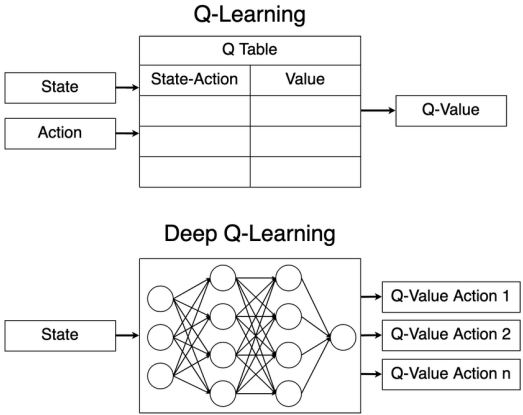
From tabular Q-learning to Deep Q-learning (visual bridge)

- Tabular: one value per (s, a).
- Deep: learn a shared model

$$Q(s, a) \approx Q_\theta(s, a).$$

- DQN typically outputs all actions:

$$Q_\theta(s, \cdot) \in \mathbb{R}^{|\mathcal{A}|}.$$



Tabular vs deep Q-learning overview.

Deep Q-Networks

Concepts • Intuition • Implementation

DRL Module 2

Deep RL in one picture: a neural network inside the agent

- In **deep RL**, the agent uses a neural network with parameters θ.

Intuition: what is θ?

- θ is the set of **trainable numbers** (weights + biases).
- Think: θ stores the agent's **knowledge** compactly; changing θ changes outputs for many states at once.
- In DQN, θ defines the mapping $s \mapsto Q_\theta(s, \cdot) \in \mathbb{R}^{|\mathcal{A}|}$.

- The network can represent:
  - a **policy** $\pi_\theta(a \mid s)$, or
  - an **action-value** $Q_\theta(s, a)$ (DQN).

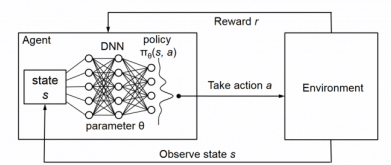


Figure: agent–environment interaction with a DNN inside the agent (provided figure).

DQN in one line: actions from Q_θ

- DQN approximates action-values with Q_θ.
- For discrete actions, output all Q-values in one forward pass:

$$Q_\theta(s, \cdot) \in \mathbb{R}^{|\mathcal{A}|}.$$

Action selection (ε-greedy during training)

$$a_t = \begin{cases} \text{random action} & \text{with prob. } \varepsilon \\ \arg\max_a Q_\theta(s_t, a) & \text{with prob. } 1 - \varepsilon \end{cases}$$

What does "$Q_\theta \approx Q^*$" mean?

- $Q^*(s, a)$ exists but is unknown.
- We choose a model class $\{Q_\theta\}$ and tune θ so that Q_θ matches Bellman-consistent targets on observed data.

Key idea

Instead of storing values, we learn θ such that the model produces good Q-values:

$$(s, a) \mapsto Q_\theta(s, a) \quad \text{or} \quad s \mapsto Q_\theta(s, \cdot).$$

Notation cheat-sheet: $\phi(s)$, $x(s)$, k, n, θ (with examples)

- $x(s) \in \mathbb{R}^n$: **input state vector** fed to the model (raw / preprocessed state).
- $\phi(s) \in \mathbb{R}^k$: **feature representation** of the state (hand-crafted or learned).
- θ: **all trainable parameters** (weights/biases) of the approximator.

Small concrete examples

- **Example A (low-dim sensors):** $x(s) = [\text{pos}, \text{vel}, \text{dist-to-goal}]^\top \in \mathbb{R}^3 \Rightarrow n = 3$. A linear feature map might be $\phi(s) = [1, \text{pos}, \text{vel}, \text{dist}]^\top \in \mathbb{R}^4 \Rightarrow k = 4$.
- **Example B (one-hot discrete state):** If there are 20 discrete states, use one-hot $x(s) \in \mathbb{R}^{20}$ $\Rightarrow n = 20$. Linear FA often uses $\phi(s) = x(s)$ so $k = n = 20$.
- **Example C (MLP features):** Suppose $x(s) \in \mathbb{R}^{10}$ (10 sensors) and we choose a hidden layer with 128 units. Then the learned feature vector is $h = \phi_\theta(s) \in \mathbb{R}^{128} \Rightarrow n = 10,\ k = 128$.

Difference between n and k

n is the dimension of the *input* $x(s)$ (set by the state encoding). k is the dimension of a *feature vector* $\phi(s)$ (chosen by us or produced by a hidden layer). In linear FA, we often set $\phi(s) = x(s)$ so $n = k$.

Linear function approximation (explicit)

- Represent state by features $\phi(s) \in \mathbb{R}^k$.
- For each action a, learn weights $\theta_a \in \mathbb{R}^k$.

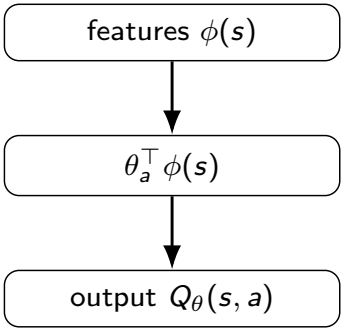
Linear Q

$$Q_\theta(s, a) = \theta_a^\top \phi(s).$$

Vector-output form

Let $W \in \mathbb{R}^{|\mathcal{A}| \times k}$ stack all $\theta_a^\top$. Then:

$$Q_\theta(s, \cdot) = W\phi(s).$$

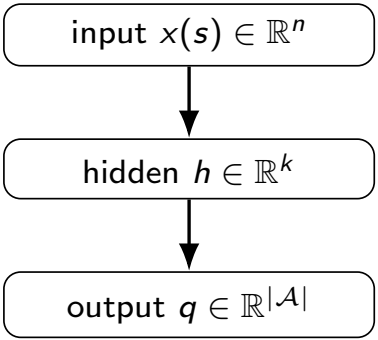


MLP approximation (where n and k appear)

- Input: $x(s) \in \mathbb{R}^n$ (state encoding; n fixed by the environment/representation).
- Choose a hidden representation size k (a design choice, e.g., $k = 128$).
- MLP forward pass (one hidden layer shown):

$$h = \sigma(W_1 x + b_1), \quad h \in \mathbb{R}^k$$

$$q = W_2 h + b_2, \quad q \in \mathbb{R}^{|\mathcal{A}|}, \quad q_a = Q_\theta(s, a).$$



Dimensions (why $\mathbb{R}^n$ and $\mathbb{R}^k$ matter)

$$W_1 \in \mathbb{R}^{k \times n}, \; b_1 \in \mathbb{R}^k, \qquad W_2 \in \mathbb{R}^{|\mathcal{A}| \times k}, \; b_2 \in \mathbb{R}^{|\mathcal{A}|}.$$

DQN target, loss, and stability

Concepts • Intuition • Implementation

DRL Module 2

DQN target + loss (core equations) + what "backprop" means

Given $(s, a, r, s', done)$:

Target (with terminal masking)

$$y = r + (1 - \text{done})\,\gamma \max_{a'} Q_{\theta^-}(s', a').$$

Loss (regress only the executed action)

$$\mathcal{L}(\theta) = \mathbb{E}\big[(y - Q_\theta(s, a))^2\big].$$

What is **backpropagation** here? (beginner explanation)

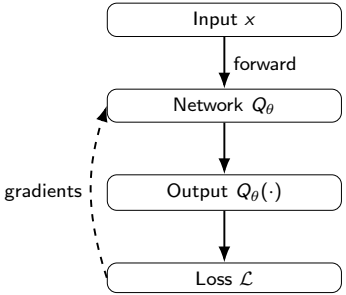
- The network output $Q_\theta(s, a)$ depends on many weights θ.
- **Backprop** is the method that computes $\nabla_\theta \mathcal{L}$: how each weight affects the loss.
- Then an optimizer (e.g., Adam) uses these gradients to update θ to reduce the loss.

Semi-gradient idea (very important)

During backprop, treat y as a constant: gradients flow only through $Q_\theta(s, a)$. (No gradient through max or through the target network Q_{θ^-}.)

Mini-primer: Backpropagation in plain language (forward → backward)

- A training step always has two parts:
  - **Forward pass:** compute prediction from the input (run the network).
  - **Backward pass (backprop):** compute gradients that tell us how to change weights to reduce the loss.
- Backprop uses the **chain rule** (layer-by-layer, from the loss backward).
- Why do we need it? A neural network may have millions of weights; backprop computes all their gradients efficiently.



Tiny example (one weight)

Suppose prediction is $\hat{y} = w\,x$ and loss is $\mathcal{L} = (y - \hat{y})^2$. Then the gradient is:

$$\frac{\partial \mathcal{L}}{\partial w} = -2\,(y - \hat{y})\,x.$$

If the prediction is too small ($y - \hat{y} > 0$) and $x > 0$, the gradient is negative $\Rightarrow$ increasing w decreases the loss.

Why a target network? (the moving-target problem)

- If we used the *same* parameters θ on both sides, the target would be:

$$y(\theta) = r + (1 - \text{done})\,\gamma \max_{a'} Q_\theta(s', a').$$

- After each optimizer step, θ changes $\Rightarrow$ both prediction and target change immediately.

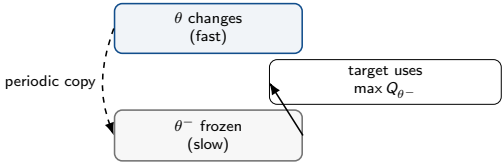
Key intuition

Training can become "chasing your own tail": the learning signal moves while you are trying to fit it, which can cause oscillation/divergence.

Fix

Use a **delayed** copy θ^- to compute the target:

$$y = r + (1 - \text{done})\,\gamma \max_{a'} Q_{\theta^-}(s', a').$$



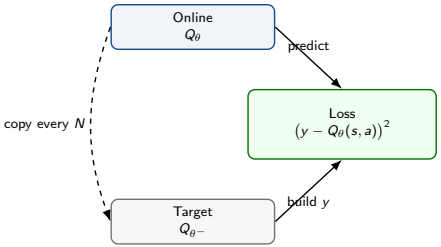
Two Q-networks in DQN: roles and "copy every N updates"

Online Q-network Q_θ (the learner)

- Used to act (ε-greedy) and for the prediction $Q_\theta(s, a)$.
- Updated every training step: **backprop computes gradients** and an optimizer (e.g., Adam) applies the update.

Target Q-network Q_{θ^-} (the stable reference)

- Used only inside the target y.
- Frozen for a while so targets change slowly.



What does "copy every N updates" mean?

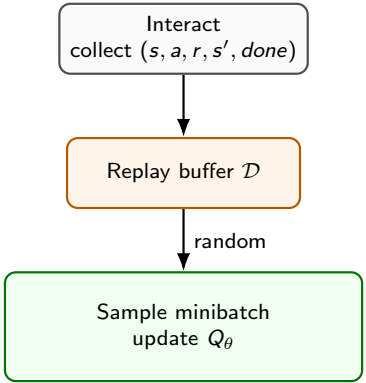
Every N training updates (or N environment steps), do a hard update:

$$\theta^- \leftarrow \theta.$$

Then keep θ^- fixed again for the next N updates.

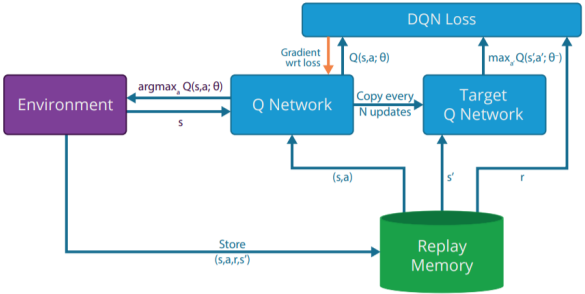
Experience replay (why it helps)

- Store transitions in a buffer $\mathcal{D}$ and sample random minibatches.
- Benefits:
  - **decorrelates** data (closer to i.i.d. assumption in SGD),
  - **reuses** experience (sample efficiency),
  - reduces variance of updates and improves stability.



DQN architecture (single diagram slide)

- Online network Q_θ: produces $Q_\theta(s, \cdot)$ and selects actions.
- Replay buffer $\mathcal{D}$: stores transitions and samples minibatches.
- Target network Q_{θ^-}: computes stable targets y.



DQN system diagram (provided figure).

One DQN update step (minibatch view): forward pass, backprop, optimizer

Given minibatch $\{(s_i, a_i, r_i, s_i', done_i)\}_{i=1}^{B}$:

1. **Forward pass (online net):** compute predictions
$$q_i = Q_\theta(s_i, a_i).$$
2. **Build targets (target net):**
$$y_i = r_i + (1 - done_i)\,\gamma \max_{a'} Q_{\theta^-}(s_i', a').$$
3. **Compute loss:**
$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B}(y_i - q_i)^2 \quad \text{(or Huber)}.$$
4. **Backward pass (backprop):** compute gradients
$$\nabla_\theta \mathcal{L}(\theta).$$
5. **Optimizer step (e.g., Adam):** update online parameters θ.
6. **Occasionally:** copy $\theta^- \leftarrow \theta$ (every N updates/steps).

Where gradients flow (the key idea students must remember)

Backprop updates θ through $q_i = Q_\theta(s_i, a_i)$ only. We treat y_i as a constant (no gradient through the max or through Q_{θ^-}).

Practice: exploration, loss, evaluation

Concepts • Intuition • Implementation

DRL Module 2

Loss choice: MSE vs Huber (one slide)

Let $\delta = y - Q_\theta(s, a)$.

MSE

$$\ell(\delta) = \delta^2$$

Sensitive to large errors early in training.

Huber (often preferred)

Quadratic near 0, linear for large errors (robust gradients).

Stability tips

Huber + smaller LR + gradient clipping often fixes divergence.

Optimizer in DQN: what is Adam (high-level, no math)?

- After backprop, we have **gradients** $\nabla_\theta \mathcal{L}$.
  - A gradient tells us *which direction* to change each weight to reduce the loss.
- An **optimizer** is the rule that turns gradients into an actual parameter update:

$$\theta \leftarrow \theta + (\text{a small change based on the gradient}).$$

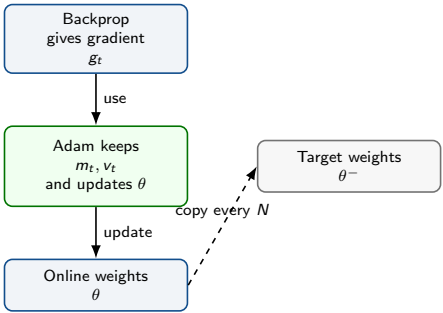
- In deep RL, gradients are often **noisy** (because targets are bootstrapped and minibatches come from replay).
- **Adam** is a popular optimizer because it automatically chooses a **reasonable step size** for each parameter.

Intuition: two ideas Adam mixes

- **Momentum (smoother direction):** instead of following today's gradient only, Adam uses a running average so updates do not zig-zag.
- **Adaptive step size (per-weight):** if a weight's gradients are usually large, Adam takes smaller steps for that weight; if they are usually small, Adam can take slightly larger steps.

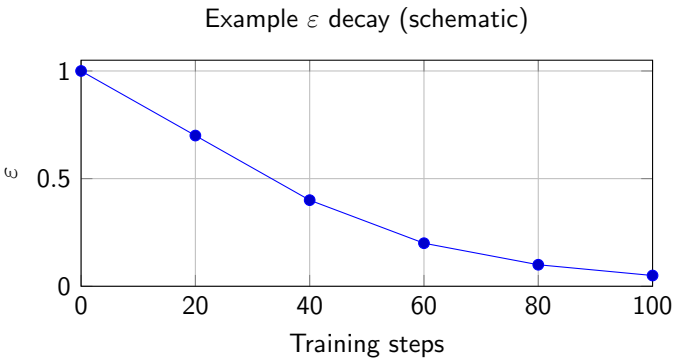
Adam in one picture: what it stores and what it updates

- Adam keeps two "memories" for each parameter:
  - m_t: **running average of gradients** (direction memory / momentum).
  - v_t: **running average of squared gradients** (size memory / scale).
- Update idea (words, not details):
  - Use m_t to decide a stable direction.
  - Divide by $\sqrt{v_t}$ so parameters with consistently large gradients move less.



Exploration: ε-greedy schedule (figure)

- Start exploratory, then decay ε to exploit.



Pitfall

Decaying too fast can prevent learning in sparse-reward tasks.

DQN pseudocode (compact)

1. Initialize online Q_θ, target $Q_{\theta^-} \leftarrow Q_\theta$, replay buffer $\mathcal{D}$.
2. Repeat:
   - choose a by ε-greedy from $Q_\theta(s, \cdot)$
   - step env, store $(s, a, r, s', done)$ in $\mathcal{D}$
   - sample minibatch from $\mathcal{D}$, compute y with Q_{θ^-}
   - compute loss and update θ with an optimizer (commonly Adam)
   - every N: copy $\theta^- \leftarrow \theta$

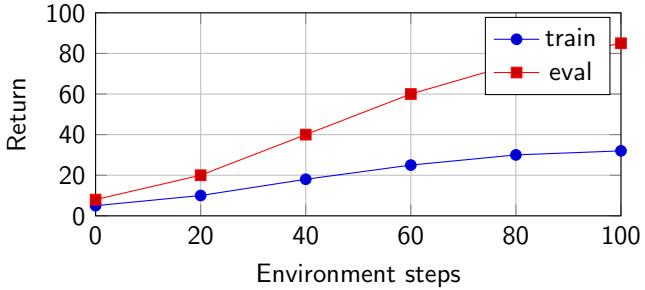
Common hyperparameters (starter values)

Parameter	Typical values
Discount γ	0.95–0.99
Learning rate (α)	10^{-4} to 10^{-3}
Optimizer	Adam (common default)
Adam ($\beta_1, \beta_2, \varepsilon$)	$(0.9, 0.999, 10^{-8})$ (typical defaults)
Batch size	32–128
Replay capacity	10^5 to 10^6
Warm-up steps	1k–10k transitions
Update frequency	every 1–4 env steps
Target copy N	1k–10k steps/updates
ε decay	$1.0 \rightarrow 0.05$ over 10^5–10^6 steps
Loss	Huber (often) or MSE

Evaluation: train vs eval (figure + guidance)

- Training return includes exploration (noisy, lower).
- Evaluation return uses greedy policy ($\varepsilon = 0$) for comparability.
- Use multiple seeds; report mean and variability.

Schematic: training vs evaluation curves



Debugging checklist (high-signal)

- Terminal masking correct? (*done* $\Rightarrow$ no bootstrap)
- Replay warm-up done before learning?
- Using Q_{θ^-} for targets (not Q_θ)?
- Optimizer settings sane?
  - learning rate too high $\Rightarrow$ divergence (Adam can still diverge)
  - try lower α, Huber loss, gradient clipping
- ε not too small early?

Summary

- DQN learns Q_θ to approximate Q^* for discrete actions.
- θ are the model's trainable weights/biases (the agent's compact knowledge).
- n = input dimension $x(s) \in \mathbb{R}^n$; k = feature/hidden dimension $\phi(s) \in \mathbb{R}^k$.
- Two networks: Q_θ learns; Q_{θ^-} stabilizes targets; copy $\theta^- \leftarrow \theta$ every N.
- Backprop computes gradients $\nabla_\theta \mathcal{L}$; Adam uses them to update θ.

References (DQN)

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.).
- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*. doi:10.1038/nature14236.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.